멀티헤드 어텐션 (Multi-Head Attention) 구현

개요

멀티헤드 어텐션은 트랜스포머 아키텍처의 핵심 혁신 중 하나입니다. 단일 어텐션 헤드 대신 여러 개의 어텐션 헤드를 병렬로 사용함으로써, 모델이 서로 다른 종류의 관계와 패턴을 동시에 학습할 수 있게 합니다.

멀티헤드 어텐션의 필요성

1. 표현 능력의 한계 극복

단일 어텐션 헤드는 하나의 관점에서만 토큰 간의 관계를 파악합니다. 예를 들어:

```
문장: "The cat sat on the mat"
```

- **구문적 관계**: "cat"과 "sat"의 주어-동사 관계
- **의미적 관계**: "cat"과 "mat"의 위치 관계
- **거리 관계**: 인접한 단어들 간의 지역적 패턴

하나의 어텐션 헤드로는 이 모든 관계를 동시에 효과적으로 포착하기 어렵습니다.

2. 다양한 패턴의 병렬 학습

멀티헤드 어텐션을 통해 각 헤드가 서로 다른 패턴을 전문화하여 학습할 수 있습니다:

- **헤드 1**: 구문적 의존성 (주어-동사, 수식어-피수식어)
- **헤드 2**: 의미적 유사성 (동의어, 반의어 관계)

- **헤드 3**: 위치적 패턴 (인접성, 거리 기반 관계)
- **헤드 4**: 장거리 의존성 (문장 전체에 걸친 관계)

멀티헤드 어텐션 동작 원리

1. 헤드 분할 (Head Splitting)

입력 차원을 헤드 수만큼 분할하여 각 헤드가 독립적으로 처리할 수 있도록 합니다.

```
# 예시: d_model=512, num_heads=8

# 각 헤드는 d_k = d_model / num_heads = 64 차원을 처리
```

2. 병렬 어텐션 계산

각 헤드에서 독립적으로 어텐션을 계산합니다:

```
Head_i = Attention(Q_i, K_i, V_i)
```

3. 결과 결합 (Concatenation)

모든 헤드의 출력을 연결하고 선형 변환을 적용합니다:

```
MultiHead(Q,K,V) = Concat(head_1, ..., head_h)W^O
```

완전한 MultiHeadAttention 클래스 구현

```
import torch
```

```python
import torch.nn as nn

import torch.nn.functional as F

import math



class MultiHeadAttention(nn.Module):

"""

멀티헤드 어텐션 구현

Args:

d_model: 모델의 차원 (임베딩 차원)

num_heads: 어텐션 헤드의 수

dropout: 드롭아웃 비율

"""

def  __init__(self, d_model, num_heads, dropout=0.1):

super(MultiHeadAttention, self).__init__()

# 차원이 헤드 수로 나누어떨어지는지 확인

assert d_model % num_heads == 0

self.d_model = d_model

self.num_heads = num_heads

self.d_k = d_model // num_heads # 각 헤드의 차원

# Query, Key, Value를 위한 선형 변환 레이어

self.w_q = nn.Linear(d_model, d_model)

self.w_k = nn.Linear(d_model, d_model)

self.w_v = nn.Linear(d_model, d_model)
```

```
# 최종 출력을 위한 선형 변환 레이어
self.w_o = nn.Linear(d_model, d_model)
# 드롭아웃
self.dropout = nn.Dropout(dropout)
def  scaled_dot_product_attention(self, query, key, value, mask=None):
"""
스케일드 닷-프로덕트 어텐션 계산
Args:
query: [batch_size, num_heads, seq_len, d_k]
key: [batch_size, num_heads, seq_len, d_k]
value: [batch_size, num_heads, seq_len, d_k]
mask: 어텐션 마스크 (옵션)
Returns:
output: [batch_size, num_heads, seq_len, d_k]
attention_weights: [batch_size, num_heads, seq_len, seq_len]
"""
d_k = query.size(-1)
# 어텐션 스코어 계산: Q * K^T / sqrt(d_k)
scores = torch.matmul(query, key.transpose(-2, -1)) / math.sqrt(d_k)
# 마스크 적용 (필요한 경우)
if mask is not None:
scores = scores.masked_fill(mask == 0, -1e9)
# 소프트맥스로 어텐션 가중치 계산
```

```
attention_weights = F.softmax(scores, dim=-1)

attention_weights = self.dropout(attention_weights)

# 가중치를 Value에 적용

output = torch.matmul(attention_weights, value)

return output, attention_weights

def  forward(self, query, key, value, mask=None):

"""

멀티헤드 어텐션 순전파

Args:

query: [batch_size, seq_len, d_model]

key: [batch_size, seq_len, d_model]

value: [batch_size, seq_len, d_model]

mask: 어텐션 마스크 (옵션)

Returns:

output: [batch_size, seq_len, d_model]

attention_weights: [batch_size, num_heads, seq_len, seq_len]

"""

batch_size, seq_len = query.size(0), query.size(1)

# 1. 선형 변환을 통해 Q, K, V 생성

Q = self.w_q(query) # [batch_size, seq_len, d_model]

K = self.w_k(key) # [batch_size, seq_len, d_model]

V = self.w_v(value) # [batch_size, seq_len, d_model]

# 2. 헤드별로 분할하여 reshape
```

```
# [batch_size, seq_len, d_model] -> [batch_size, seq_len, num_heads, d_k]
Q = Q.view(batch_size, seq_len, self.num_heads, self.d_k)
K = K.view(batch_size, seq_len, self.num_heads, self.d_k)
V = V.view(batch_size, seq_len, self.num_heads, self.d_k)
# 3. 차원 순서 변경: [batch_size, num_heads, seq_len, d_k]
Q = Q.transpose(1, 2)
K = K.transpose(1, 2)
V = V.transpose(1, 2)
# 4. 각 헤드에서 어텐션 계산
attention_output, attention_weights = self.scaled_dot_product_attention(
Q, K, V, mask
)
# 5. 헤드들을 다시 연결
# [batch_size, num_heads, seq_len, d_k] -> [batch_size, seq_len, num_heads, d_k]
attention_output = attention_output.transpose(1, 2)
# [batch_size, seq_len, num_heads, d_k] -> [batch_size, seq_len, d_model]
attention_output = attention_output.contiguous().view(
batch_size, seq_len, self.d_model
)
# 6. 최종 선형 변환
output = self.w_o(attention_output)
return output, attention_weights
```

```python
# 사용 예시
def  demonstrate_multihead_attention():
"""멀티헤드 어텐션 사용 예시"""
# 하이퍼파라미터 설정
batch_size = 2
seq_len = 10
d_model = 512
num_heads = 8
# 멀티헤드 어텐션 모델 생성
mha = MultiHeadAttention(d_model, num_heads)
# 더미 입력 데이터 생성
x = torch.randn(batch_size, seq_len, d_model)
# 셀프 어텐션 수행 (Q, K, V 모두 동일한 입력)
output, attention_weights = mha(x, x, x)
print(f"입력 크기: {x.shape}")
print(f"출력 크기: {output.shape}")
print(f"어텐션 가중치 크기: {attention_weights.shape}")
return output, attention_weights


if __name__ == "__main__":
output, weights = demonstrate_multihead_attention()
```

헤드별 패턴 분석 예시

다음은 실제 훈련된 모델에서 각 헤드가 학습하는 서로 다른 패턴의 예시입니다:

1. 구문적 패턴을 학습하는 헤드

```
def analyze_syntactic_patterns():
"""구문적 관계를 학습한 헤드의 어텐션 패턴 분석"""
sentence = ["The", "quick", "brown", "fox", "jumps", "over", "the", "lazy", "dog
# 예시: 헤드 1이 학습한 패턴
# 주어-동사 관계에 높은 어텐션 가중치
syntactic_attention = {
"fox": {"jumps": 0.8}, # 주어-동사
"quick": {"fox": 0.7}, # 형용사-명사
"brown": {"fox": 0.6}, # 형용사-명사
"lazy": {"dog": 0.9} # 형용사-명사
}
return syntactic_attention
```

2. 의미적 유사성을 학습하는 헤드

```
def analyze_semantic_patterns():
"""의미적 관계를 학습한 헤드의 어텐션 패턴 분석"""
# 예시: 헤드 2가 학습한 패턴
```

```
# 의미적으로 관련된 단어들 간의 높은 어텐션
semantic_attention = {
"fox": {"dog": 0.6}, # 동물 간의 관계
"quick": {"lazy": 0.4}, # 반의어 관계
"jumps": {"over": 0.7} # 동작과 방향의 관계
}
return semantic_attention
```

3. 위치적 패턴을 학습하는 헤드

```
def  analyze_positional_patterns():
"""위치적 관계를 학습한 헤드의 어텐션 패턴 분석"""
# 예시: 헤드 3이 학습한 패턴
# 인접한 토큰들에 높은 어텐션
positional_attention = {
"The": {"quick": 0.8}, # 인접 토큰
"quick": {"brown": 0.9}, # 인접 토큰
"brown": {"fox": 0.8}, # 인접 토큰
}
return positional_attention
```

어텐션 가중치 시각화

```python
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np


def  visualize_attention_heads(attention_weights, tokens, head_idx=0):
"""
특정 헤드의 어텐션 가중치를 히트맵으로 시각화
Args:
attention_weights: [num_heads, seq_len, seq_len]
tokens: 토큰 리스트
head_idx: 시각화할 헤드 인덱스
"""
# 특정 헤드의 어텐션 가중치 추출
head_attention = attention_weights[head_idx].detach().numpy()
# 히트맵 생성
plt.figure(figsize=(10, 8))
sns.heatmap(
head_attention,
xticklabels=tokens,
yticklabels=tokens,
cmap='Blues',
annot=True,
```

```
fmt='.2f'
)
plt.title(f'Head {head_idx + 1} Attention Weights')
plt.xlabel('Key (Attended to)')
plt.ylabel('Query (Attending from)')
plt.tight_layout()
plt.show()


def  compare_multiple_heads(attention_weights, tokens, num_heads_to_show=4):
"""여러 헤드의 어텐션 패턴을 동시에 비교"""
fig, axes = plt.subplots(2, 2, figsize=(15, 12))
axes = axes.flatten()
for i in  range(min(num_heads_to_show, len(axes))):
head_attention = attention_weights[i].detach().numpy()
sns.heatmap(
head_attention,
xticklabels=tokens,
yticklabels=tokens,
cmap='Blues',
ax=axes[i],
cbar=True
)
axes[i].set_title(f'Head {i + 1}')
```

```
axes[i].set_xlabel('Key')

axes[i].set_ylabel('Query')

plt.tight_layout()

plt.show()
```

실제 사용 예시와 성능 분석

```
def performance_comparison():

"""단일 헤드 vs 멀티헤드 어텐션 성능 비교"""

# 단일 헤드 어텐션

class SingleHeadAttention(nn.Module):

def __init__(self, d_model):

super().__init__()

self.d_model = d_model

self.w_q = nn.Linear(d_model, d_model)

self.w_k = nn.Linear(d_model, d_model)

self.w_v = nn.Linear(d_model, d_model)

def forward(self, x):

Q = self.w_q(x)

K = self.w_k(x)

V = self.w_v(x)

scores = torch.matmul(Q, K.transpose(-2, -1)) / math.sqrt(self.d_model)

attention = F.softmax(scores, dim=-1)
```

```
output = torch.matmul(attention, V)

return output

# 성능 비교 실험

d_model = 512

seq_len = 100

batch_size = 32

# 모델 생성

single_head = SingleHeadAttention(d_model)

multi_head = MultiHeadAttention(d_model, num_heads=8)

# 더미 데이터

x = torch.randn(batch_size, seq_len, d_model)

# 파라미터 수 비교

single_params = sum(p.numel() for p in single_head.parameters())

multi_params = sum(p.numel() for p in multi_head.parameters())

print(f"단일 헤드 파라미터 수: {single_params:,}")

print(f"멀티헤드 파라미터 수: {multi_params:,}")

print(f"파라미터 증가율: {multi_params/single_params:.2f}x")

# 계산 시간 비교

import time

# 단일 헤드 시간 측정

start_time = time.time()

for _ in  range(100):

_ = single_head(x)
```

```
single_time = time.time() - start_time

# 멀티헤드 시간 측정

start_time = time.time()

for _ in  range(100):

_ = multi_head(x, x, x)

multi_time = time.time() - start_time

print(f"단일 헤드 실행 시간: {single_time:.4f}초")

print(f"멀티헤드 실행 시간: {multi_time:.4f}초")

print(f"시간 증가율: {multi_time/single_time:.2f}x")



if __name__ == "__main__":

performance_comparison()
```

핵심 개념 정리

1. 멀티헤드 어텐션의 장점

- **다양한 관계 포착**: 각 헤드가 서로 다른 종류의 관계를 학습
- **병렬 처리**: 여러 헤드를 동시에 계산하여 효율성 향상
- **표현력 증대**: 단일 헤드보다 풍부한 표현 학습 가능
- **안정적 학습**: 여러 헤드의 앙상블 효과로 학습 안정성 향상

2. 구현 시 주의사항

- **차원 일치**: `d_model` 이 `num_heads` 로 나누어떨어져야 함

- **메모리 효율성**: 헤드 수가 증가하면 메모리 사용량도 증가
- **계산 복잡도**: 헤드 수에 비례하여 계산량 증가
- **하이퍼파라미터 튜닝**: 헤드 수는 모델 성능에 큰 영향을 미침

3. 실무 적용 팁

- **일반적인 헤드 수**: 8개 또는 16개가 많이 사용됨
- **작은 모델**: 4-6개 헤드로 시작
- **큰 모델**: 12-16개 헤드 사용
- **태스크별 조정**: 복잡한 태스크일수록 더 많은 헤드 필요

다음 단계

멀티헤드 어텐션을 이해했다면, 다음 문서에서는 이를 활용한 **인코더 아키텍처**의 전체 구조를 살펴보겠습니다. 인코더에서 멀티헤드 어텐션이 어떻게 피드포워드 네트워크, 잔차 연결, 레이어 정규화와 결합되는지 알아보겠습니다.

네비게이션

- **이전**: 포지셔널 인코딩
- **다음**: 인코더 아키텍처
- **목차**: README.md
- **전체 목록**: 네비게이션

관련 문서:

- 기초: 어텐션 메커니즘